

Network Traffic Analysis Using Wireshark

Prepared By:

Issac George

Project Type:

Self-Initiated Hands-on Project

Domain:

Cybersecurity

Date:

January 2026

Tools Used:

Kali Linux, Ubuntu, Wireshark,

Nmap, hping3, VirtualBox

Introduction

Network traffic analysis is a critical skill in cybersecurity, particularly for detecting malicious activities and understanding how attacks manifest at the packet level. Tools such as Wireshark enable security professionals to capture and analyze network traffic in real time, providing deep visibility into network communications and potential threats.

This project focuses on analyzing network traffic within a controlled virtual lab environment using Wireshark. Kali Linux was used as a simulated attacker system, while Ubuntu Linux served as the target system. Activities such as Nmap-based network scanning and TCP SYN flood traffic were generated, and the resulting packets were captured and analyzed to study attack patterns and network behaviour.

The objective of this project is to gain hands-on experience in identifying different types of network-based threats and understanding how they appear in real network traffic from a defensive and analytical perspective.

Objectives

- Understand the fundamentals of network traffic analysis using Wireshark.
- Capture and analyze live network packets in a controlled lab environment.
- Observe and analyze Nmap-based network scanning at the packet level.
- Study the behaviour of TCP SYN flood traffic and its impact on network communication.
- Identify common attack patterns and abnormal traffic characteristics.
- Gain hands-on experience relevant to defensive cybersecurity and SOC analysis.

Lab Setup and Environment

A controlled virtual lab environment was created to perform and analyze **simulated network attacks** safely. Oracle VirtualBox was used as the virtualization platform to host both the attacker and target systems.

The attacker machine was **Kali Linux**, which was used to generate simulated network attack traffic and scanning activity. The target system was **Ubuntu Linux**, which served as the **target system** for network traffic analysis. Both virtual machines were configured using **Bridged Adapter** networking mode, allowing them to communicate directly on the same network.

Wireshark was used on the attacker machine to capture and analyze network traffic in real time. The lab setup enabled the observation of packet-level behaviour for different types of network activity, including normal communication and simulated attack traffic, in an isolated and controlled environment.

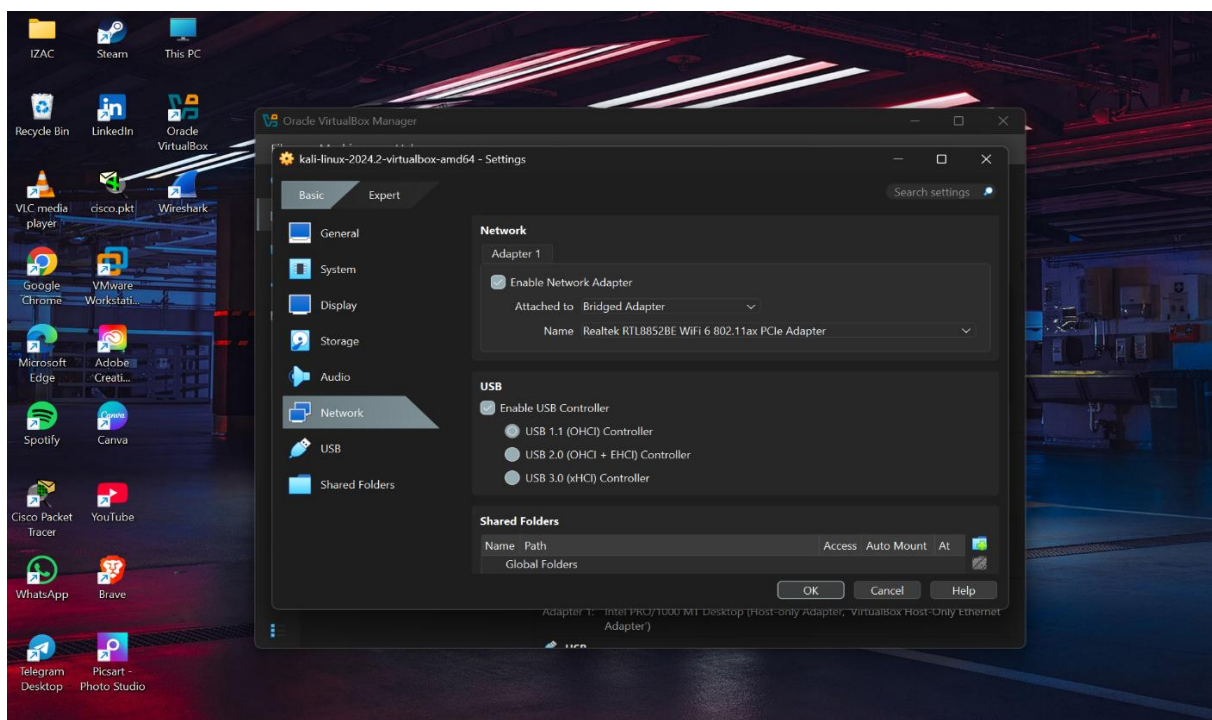


Figure 1: Network adapter configuration of Kali Linux virtual machine using Bridged Adapter mode

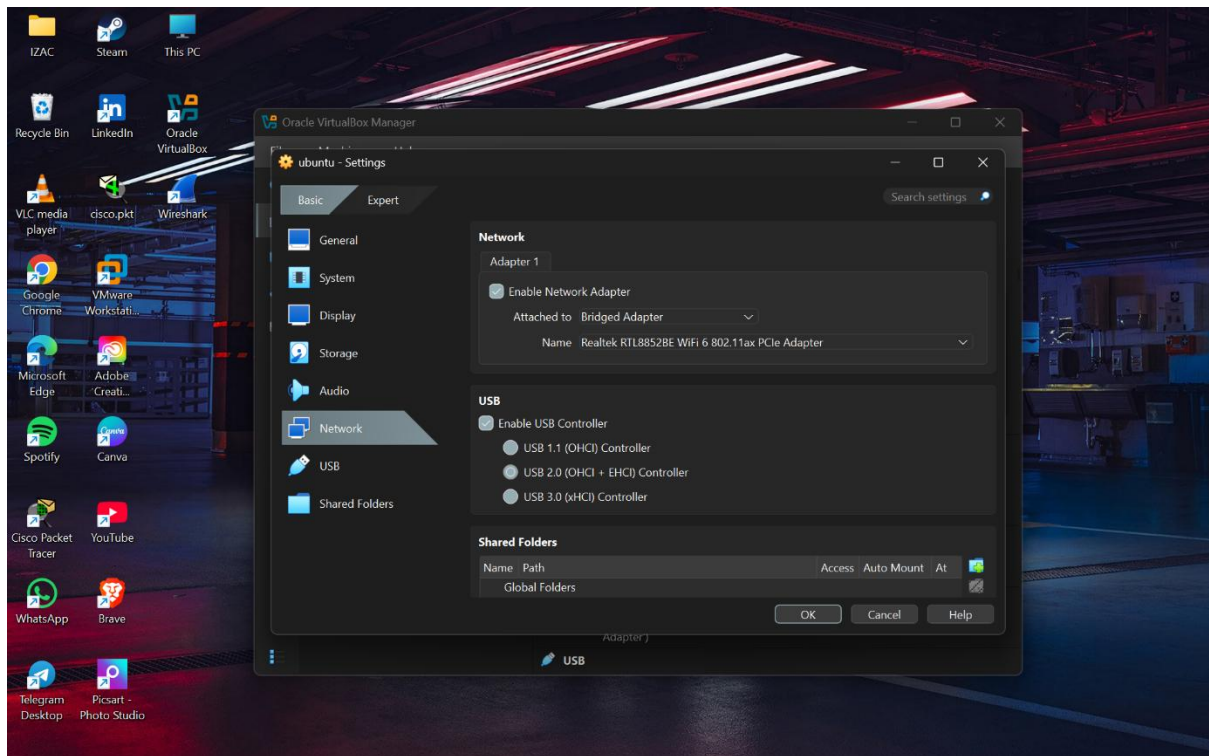
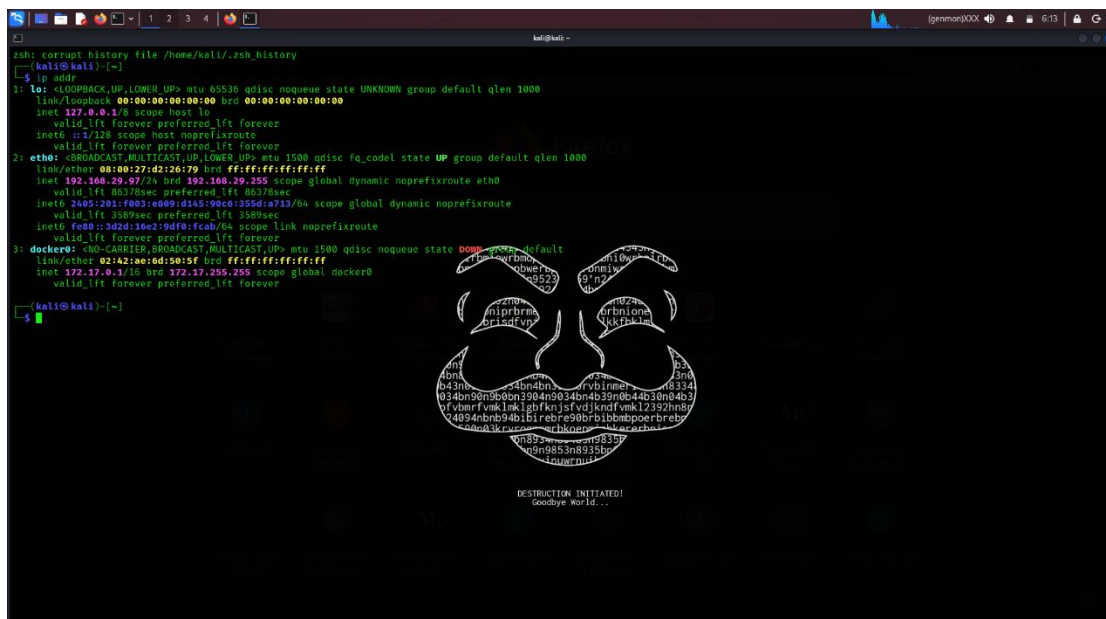


Figure 2: Network adapter configuration of Ubuntu Linux virtual machine using Bridged Adapter mode

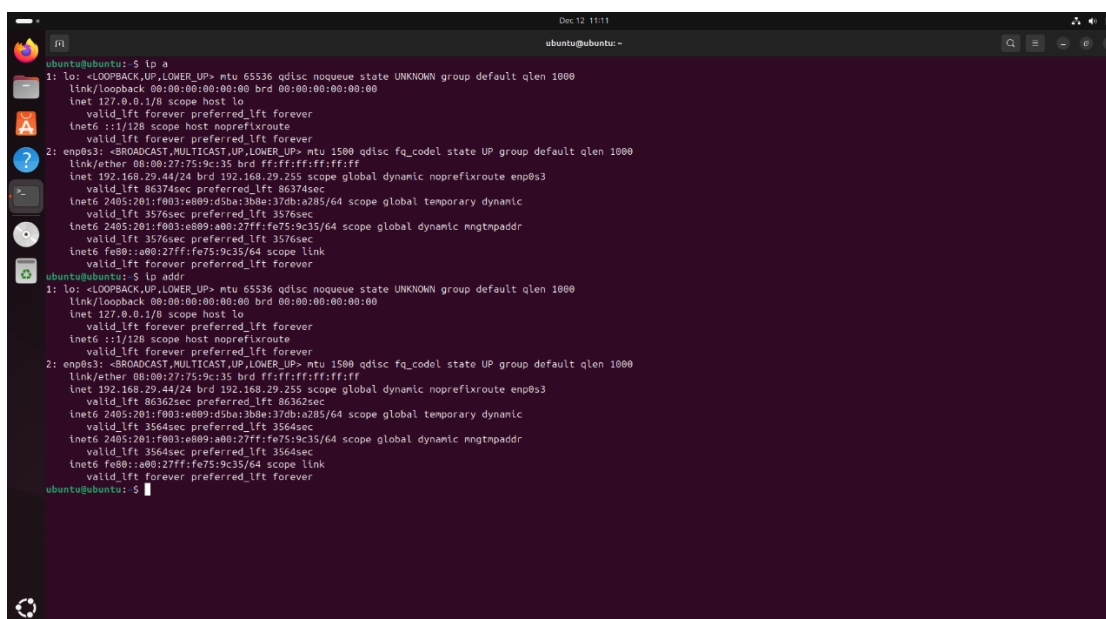
IP Address Verification

After configuring the virtual machines with bridged networking, the IP addresses of both systems were verified to ensure proper network connectivity. This step confirmed that both Kali Linux and Ubuntu Linux were assigned IP addresses within the same network subnet, allowing direct communication between the systems for traffic generation and analysis.



```
zsh: corrupt history file /home/kali/.zsh_history
kali@kali:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:d2:26:79 brd ff:ff:ff:ff:ff:ff
    inet 192.168.29.57/24 brd 192.168.29.255 scope global dynamic noprefixroute eth0
        valid_lft 86378sec preferred_lft 86378sec
    inet6 2405:201:f003:e009:d165:90c6:355d:a713/64 scope global dynamic noprefixroute
        valid_lft 3589sec preferred_lft 3589sec
    inet6 fe80::a00:27ff:fe75:9c35/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:ac:1d:50:1f brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
kali@kali:~$
```

Figure 3: IP address configuration of Kali Linux virtual machine



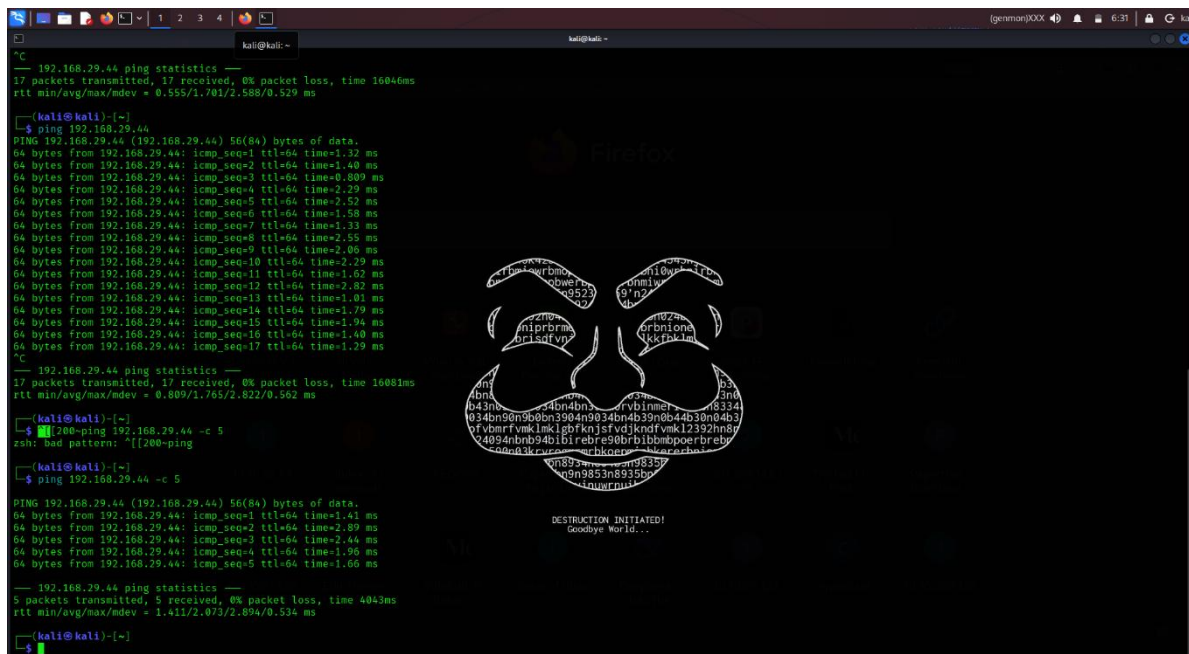
```
Dec 12 11:11
ubuntu@ubuntu:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:75:9c:35 brd ff:ff:ff:ff:ff:ff
    inet 192.168.29.44/24 brd 192.168.29.255 scope global dynamic noprefixroute enp0s3
        valid_lft 86362sec preferred_lft 86362sec
    inet6 2405:201:f003:e009:d5ba:3b8e:37db:a285/64 scope global temporary dynamic
        valid_lft 3576sec preferred_lft 3576sec
    inet6 2405:201:f003:e009:a00:27ff:fe75:9c35/64 scope global dynamic mngtnpaddr
        valid_lft 3576sec preferred_lft 3576sec
    inet6 fe80::a00:27ff:fe75:9c35/64 scope link
        valid_lft forever preferred_lft forever
ubuntu@ubuntu:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:75:9c:35 brd ff:ff:ff:ff:ff:ff
    inet 192.168.29.44/24 brd 192.168.29.255 scope global dynamic noprefixroute enp0s3
        valid_lft 86362sec preferred_lft 86362sec
    inet6 2405:201:f003:e009:d5ba:3b8e:37db:a285/64 scope global temporary dynamic
        valid_lft 3564sec preferred_lft 3564sec
    inet6 2405:201:f003:e009:a00:27ff:fe75:9c35/64 scope global dynamic mngtnpaddr
        valid_lft 3564sec preferred_lft 3564sec
    inet6 fe80::a00:27ff:fe75:9c35/64 scope link
        valid_lft forever preferred_lft forever
ubuntu@ubuntu:~$
```

Figure 4: IP address configuration of Ubuntu Linux virtual machine

ICMP Traffic Analysis

Internet Control Message Protocol (ICMP) is commonly used to test network connectivity between hosts. In this experiment, ICMP echo requests were sent from the Kali Linux system to the Ubuntu Linux system using the ping command.

While generating ICMP traffic, Wireshark was used on the Kali Linux machine to capture and analyze the packets in real time. This allowed observation of ICMP echo requests and echo replies at the packet level.



```

kali@kali: ~
└─$ ping 192.168.29.44
PING 192.168.29.44 (192.168.29.44) 56(84) bytes of data:
64 bytes from 192.168.29.44: icmp_seq=1 ttl=64 time=1.32 ms
64 bytes from 192.168.29.44: icmp_seq=2 ttl=64 time=1.40 ms
64 bytes from 192.168.29.44: icmp_seq=3 ttl=64 time=0.809 ms
64 bytes from 192.168.29.44: icmp_seq=4 ttl=64 time=2.29 ms
64 bytes from 192.168.29.44: icmp_seq=5 ttl=64 time=2.52 ms
64 bytes from 192.168.29.44: icmp_seq=6 ttl=64 time=1.58 ms
64 bytes from 192.168.29.44: icmp_seq=7 ttl=64 time=1.33 ms
64 bytes from 192.168.29.44: icmp_seq=8 ttl=64 time=2.55 ms
64 bytes from 192.168.29.44: icmp_seq=9 ttl=64 time=2.00 ms
64 bytes from 192.168.29.44: icmp_seq=10 ttl=64 time=2.29 ms
64 bytes from 192.168.29.44: icmp_seq=11 ttl=64 time=1.62 ms
64 bytes from 192.168.29.44: icmp_seq=12 ttl=64 time=2.82 ms
64 bytes from 192.168.29.44: icmp_seq=13 ttl=64 time=1.01 ms
64 bytes from 192.168.29.44: icmp_seq=14 ttl=64 time=1.79 ms
64 bytes from 192.168.29.44: icmp_seq=15 ttl=64 time=1.94 ms
64 bytes from 192.168.29.44: icmp_seq=16 ttl=64 time=1.40 ms
64 bytes from 192.168.29.44: icmp_seq=17 ttl=64 time=1.29 ms
--- 192.168.29.44 ping statistics ---
17 packets transmitted, 17 received, 0% packet loss, time 16046ms
rtt min/avg/max/mdev = 0.555/1.701/2.588/0.529 ms

kali@kali: ~
└─$ ping 192.168.29.44 -c 5
PING 192.168.29.44 (192.168.29.44) 56(84) bytes of data:
64 bytes from 192.168.29.44: icmp_seq=1 ttl=64 time=1.41 ms
64 bytes from 192.168.29.44: icmp_seq=2 ttl=64 time=2.89 ms
64 bytes from 192.168.29.44: icmp_seq=3 ttl=64 time=1.44 ms
64 bytes from 192.168.29.44: icmp_seq=4 ttl=64 time=1.96 ms
64 bytes from 192.168.29.44: icmp_seq=5 ttl=64 time=1.66 ms
--- 192.168.29.44 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4043ms
rtt min/avg/max/mdev = 1.411/2.073/2.894/0.534 ms

kali@kali: ~
└─$
```

ASCII art skull:

```

      .--.  .--.  .--.  .--.  .--.  .--.  .--.  .--.  .--.  .--.
     /  /  /  /  /  /  /  /  /  /  /  /  /  /  /  /  /  /
    /  /  /  /  /  /  /  /  /  /  /  /  /  /  /  /  /  /
   /  /  /  /  /  /  /  /  /  /  /  /  /  /  /  /  /  /
  /  /  /  /  /  /  /  /  /  /  /  /  /  /  /  /  /  /
 /  /  /  /  /  /  /  /  /  /  /  /  /  /  /  /  /  /
/  /  /  /  /  /  /  /  /  /  /  /  /  /  /  /  /  /
 \  \  \  \  \  \  \  \  \  \  \  \  \  \  \  \  \  \
  \  \  \  \  \  \  \  \  \  \  \  \  \  \  \  \  \  \
   \  \  \  \  \  \  \  \  \  \  \  \  \  \  \  \  \  \
    \  \  \  \  \  \  \  \  \  \  \  \  \  \  \  \  \  \
     \  \  \  \  \  \  \  \  \  \  \  \  \  \  \  \  \
      --  --  --  --  --  --  --  --  --  --  --  --

```

DESTRUCTION INITIATED!
Goodbye World...

Figure 5: ICMP echo requests sent from Kali Linux to Ubuntu Linux using ping command

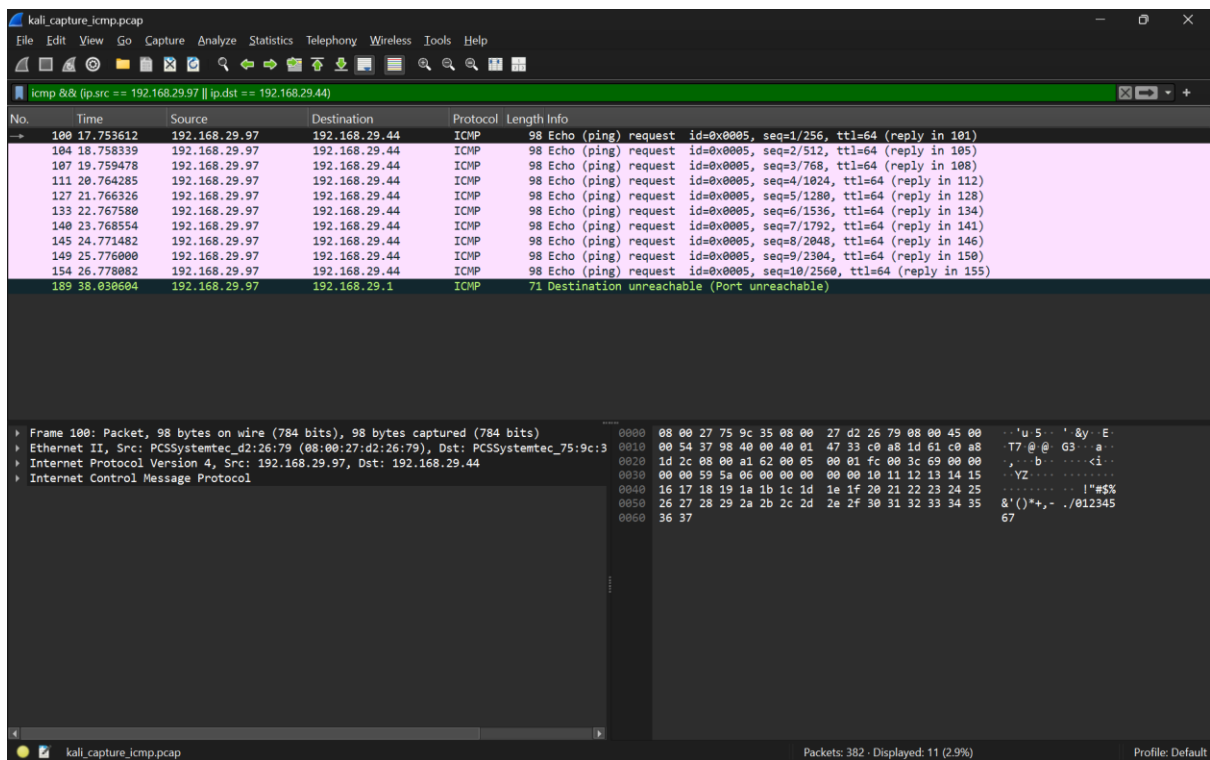


Figure 6: ICMP traffic captured in Wireshark during ping operation

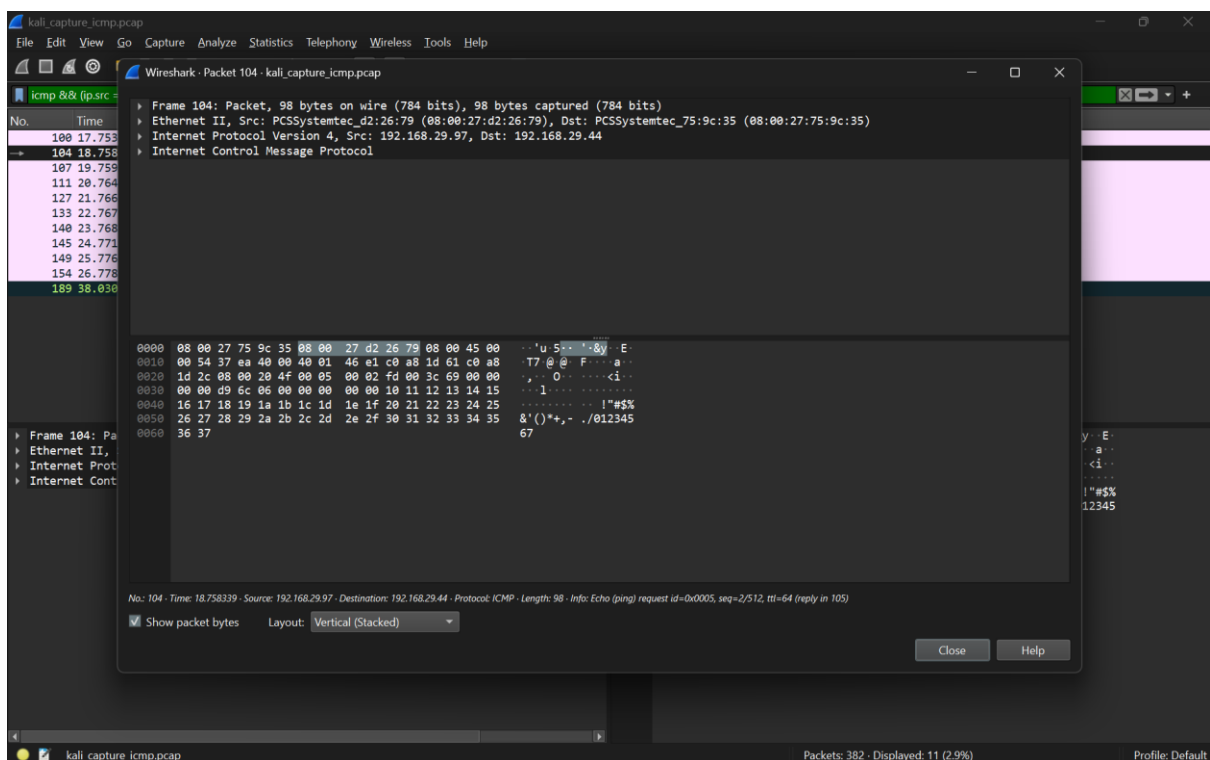


Figure 7: Detailed ICMP packet structure observed in Wireshark

Analysis Summary

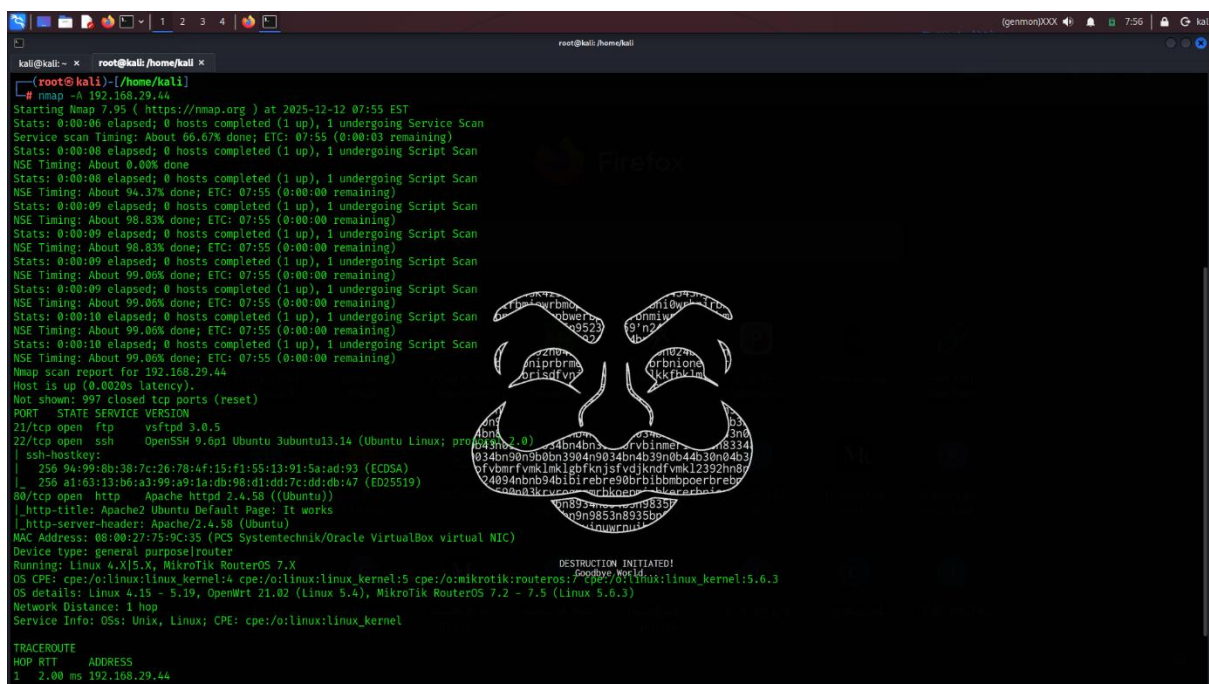
The captured packets show ICMP echo request and echo reply messages exchanged between the two systems. Source and destination IP addresses confirm communication between Kali Linux and Ubuntu Linux. Packet details such as sequence number, TTL value, and frame length were analyzed to understand ICMP behaviour at the protocol level.

Simulated Nmap Network Scanning Attack

Nmap Aggressive Scan Execution

To simulate a reconnaissance-based network attack, an Nmap aggressive scan (-A) was executed from the Kali Linux system against the Ubuntu Linux target system. The -A option enables multiple scanning techniques, including operating system detection, service and version enumeration, default script scanning, and traceroute.

This type of scan represents an advanced reconnaissance phase commonly performed by attackers to gather detailed information about a target system before launching further attacks.



```
(root@kali) [/home/kali]
# nmap -A 192.168.29.44
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-12 07:55 EST
Stats: 0:00:06 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 66.67% done; ETC: 07:55 (0:00:03 remaining)
Stats: 0:00:08 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 0.00% done
Stats: 0:00:08 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 94.37% done; ETC: 07:55 (0:00:00 remaining)
Stats: 0:00:09 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 98.83% done; ETC: 07:55 (0:00:00 remaining)
Stats: 0:00:09 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 98.83% done; ETC: 07:55 (0:00:00 remaining)
Stats: 0:00:09 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 99.00% done; ETC: 07:55 (0:00:00 remaining)
Stats: 0:00:10 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 99.00% done; ETC: 07:55 (0:00:00 remaining)
Stats: 0:00:10 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 99.00% done; ETC: 07:55 (0:00:00 remaining)
Nmap scan report for 192.168.29.44
Host is up (0.0020s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 3.0.5
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu Jubuntu13.14 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|_ 256 9a:99:8b:38:7c:26:78:4f:15:f1:55:13:91:5a:ad:93 (ECDSA)
|_ 256 a1:63:13:b6:a3:99:a9:1a:db:98:d1:dd:7c:dd:db:47 (ED25519)
80/tcp    open  http     Apache httpd 2.4.58 ((Ubuntu))
|_ http-title: Apache2 Ubuntu Default Page: It works
|_ http-server-header: Apache/2.4.58 (Ubuntu)
MAC Address: 08:00:27:75:9C:35 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Device type: general purpose/router
Running: Linux 4.X15.X, Mikrotik RouterOS 7.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:mikrotik:routeros:7
OS details: Linux 4.15 - 5.19, OpenWrt 21.02 (Linux 5.4), Mikrotik RouterOS 7.2 - 7.5 (Linux 5.6.3)
Network Distance: 1 hop
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE
HOP RTT ADDRESS
1 2.00 ms 192.168.29.44
```

Figure 8: Nmap aggressive scan (-A) executed from Kali Linux against the target system

TCP SYN Scan Traffic Analysis

During the aggressive scan, Wireshark captured a significant number of TCP SYN packets sent from the Kali Linux machine to various ports on the Ubuntu target system. These SYN packets are part of Nmap's port scanning mechanism, where connection attempts are initiated to determine whether ports are open, closed, or filtered.

The high frequency of SYN packets targeting multiple ports is a clear indicator of scanning activity rather than normal user traffic.

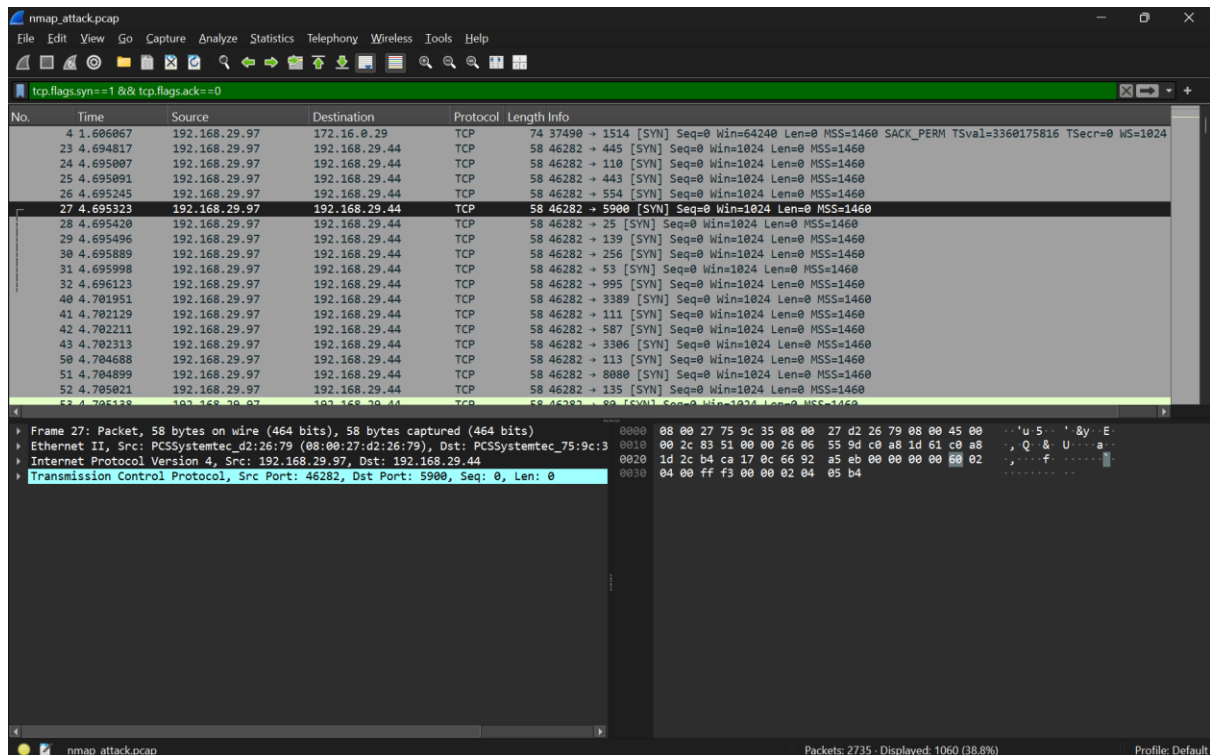


Figure 9: TCP SYN packets generated during Nmap aggressive scan

Target System Responses (RST/ACK and SYN/ACK)

Analysis of the captured packets showed that the target system responded with **TCP RST/ACK packets** for closed ports, indicating rejection of connection attempts. For open ports, **SYN/ACK responses** were observed, confirming active services on the target system.

This exchange of SYN requests and RST/ACK or SYN/ACK responses allows Nmap to accurately identify port states during reconnaissance.

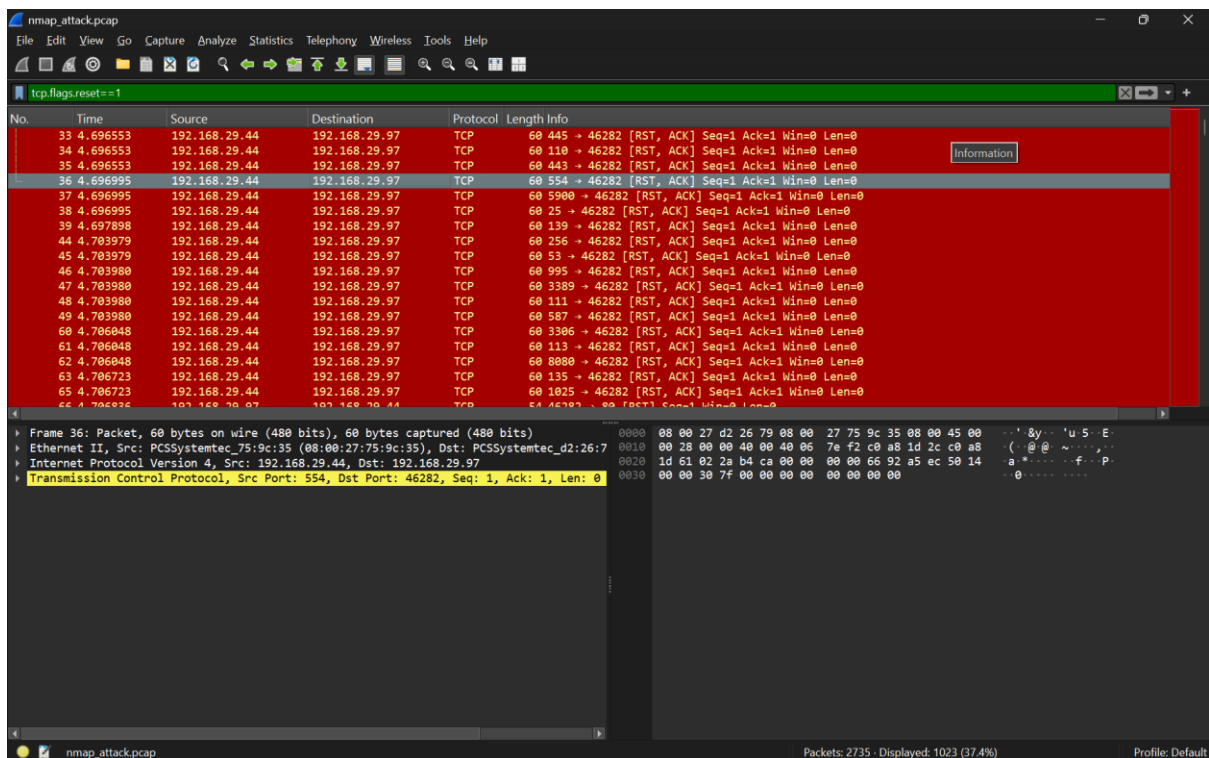


Figure 10: TCP RST/ACK responses from the target system during Nmap scan

Service and OS Detection Evidence

Because the -A scan includes service and operating system detection, application-layer packets were observed in the capture. Protocol-specific responses such as FTP and SSH banners were identified, revealing service versions and confirming the presence of active services on the target system.

These findings matched the output of the Nmap scan and demonstrate how detailed system information can be extracted through network reconnaissance.

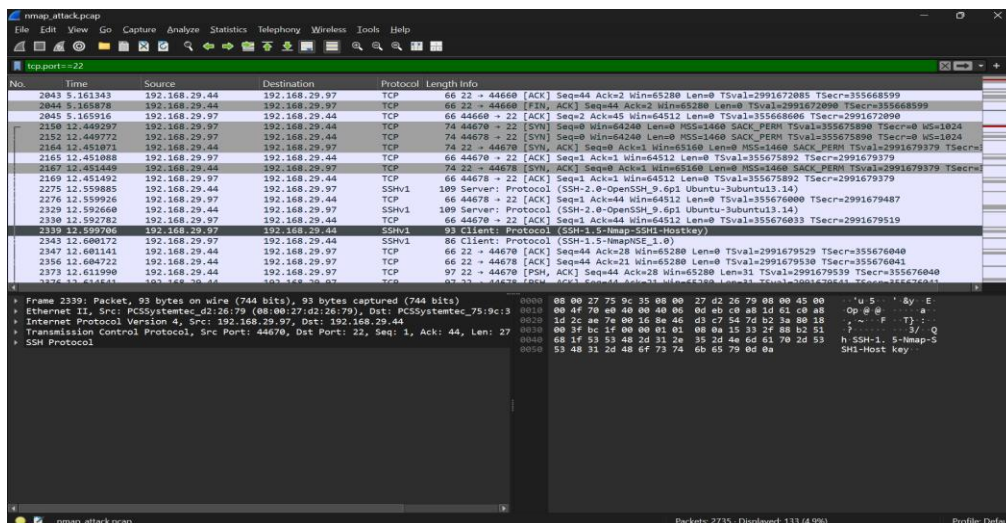


Figure 11: SSH service and version information captured during Nmap aggressive scan

Analysis Summary

The captured network traffic clearly demonstrates the behaviour of an **Nmap aggressive reconnaissance scan**. The presence of repeated TCP SYN packets, combined with RST/ACK and SYN/ACK responses, confirms systematic port scanning activity. Additionally, service-level responses highlight how attackers can extract detailed information about running services and operating systems.

This analysis emphasizes the importance of network traffic monitoring and intrusion detection mechanisms to identify reconnaissance activity early and prevent further exploitation attempts.

TCP SYN Flood Attack Analysis

During this phase of the experiment, a TCP SYN flood attack was simulated from the Kali Linux system targeting the Ubuntu Linux system. The attack was generated using the hping3 tool, which allows crafting and sending custom TCP packets at high speed.

The SYN flood attack involved sending a continuous stream of TCP SYN packets to port 80 of the target system without completing the TCP three-way handshake. This behaviour is intended to exhaust server resources by forcing it to maintain half-open connections.

Wireshark was used to capture and analyze the network traffic during the attack. The captured packets show an abnormally high volume of TCP SYN packets originating from the attacker's IP address and directed toward the target system. The absence of corresponding ACK packets confirms that the connections were never completed, which is a defining characteristic of a SYN flood attack.

Additionally, Wireshark highlights repeated TCP port reuse and identical packet structures, further indicating automated flooding behaviour rather than legitimate client connections.

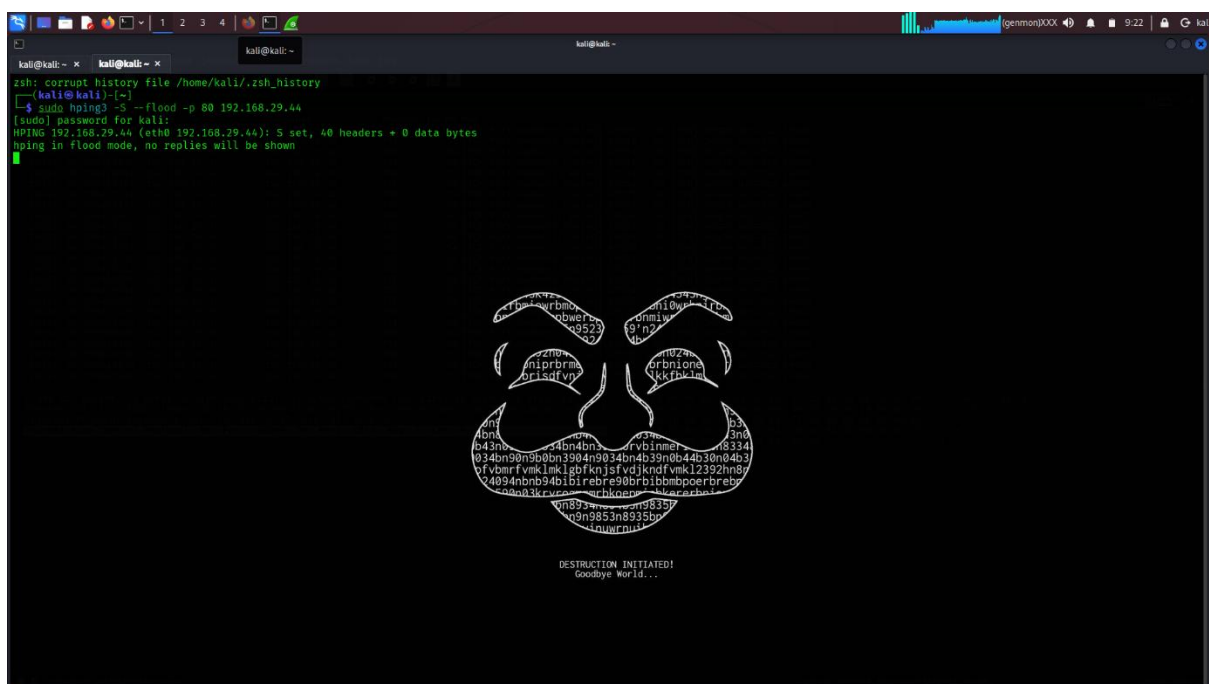


Figure 12: Execution of TCP SYN flood attack using hping3 from Kali Linux targeting Ubuntu system

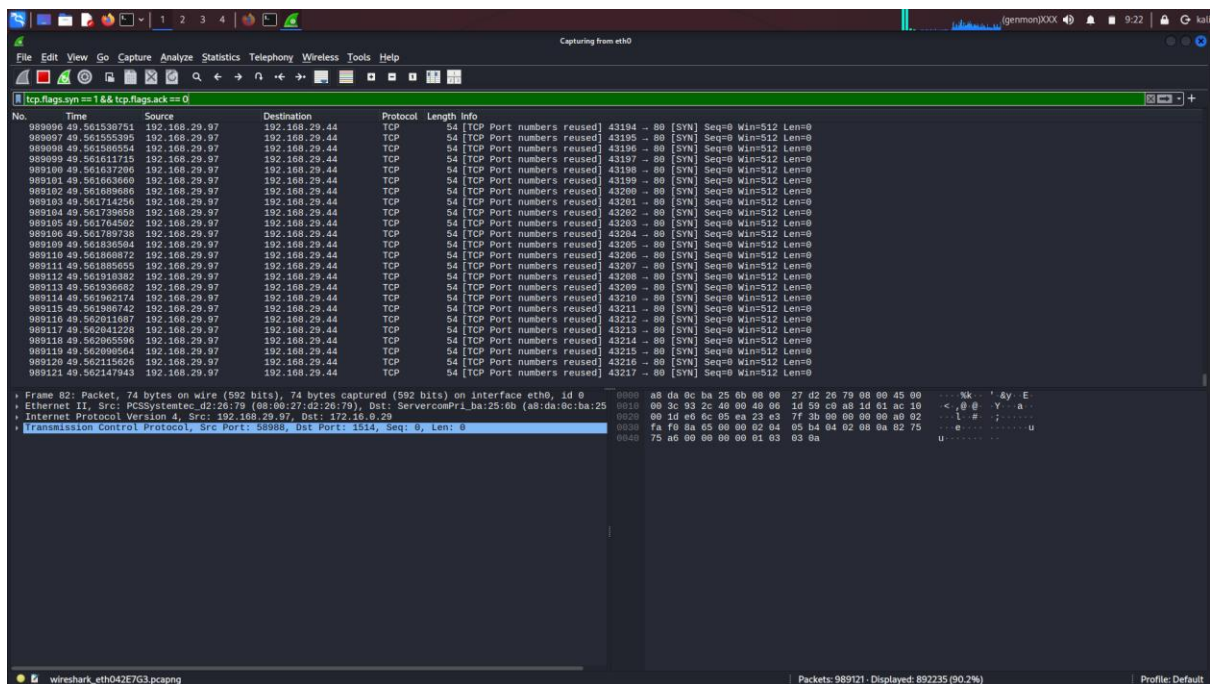


Figure 13: Wireshark capture showing a high volume of TCP SYN packets during SYN flood attack

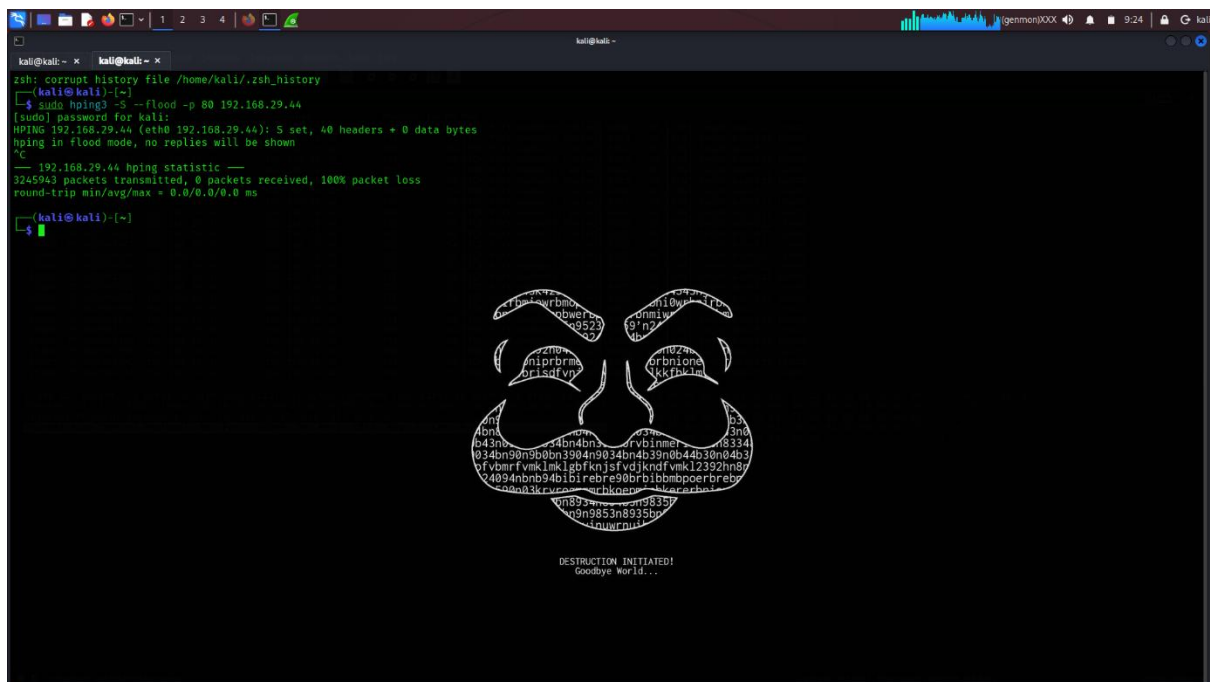


Figure 14: hping3 statistics indicating successful SYN flood traffic generation

Analysis Summary

The captured network traffic clearly demonstrates the behaviour of a TCP SYN flood attack. The excessive number of SYN packets without successful handshake completion confirms an attempt to overwhelm the target system's connection-handling capacity. Such traffic patterns are commonly associated with denial-of-service (DoS) attacks and can significantly degrade service availability.

This analysis highlights how SYN flood attacks appear at the packet level and emphasizes the importance of network monitoring, rate limiting, and intrusion detection systems to detect and mitigate denial-of-service attacks in real-world environments.

Conclusion

This project successfully demonstrated the importance of network traffic analysis in identifying and understanding different types of network activities and simulated attacks using Wireshark. By conducting the experiment in a controlled virtual lab environment, realistic network behaviour was safely generated and analyzed without impacting real-world systems.

Through ICMP traffic analysis, normal network communication between Kali Linux and Ubuntu Linux was observed and verified. The captured ICMP echo request and reply packets provided insight into basic connectivity testing and protocol-level behaviour.

The Nmap aggressive scan highlighted how reconnaissance activities appear at the packet level. Repeated TCP SYN packets, along with SYN/ACK and RST/ACK responses, revealed systematic port scanning and service enumeration techniques used by attackers to gather information about a target system.

Finally, the simulated TCP SYN flood attack demonstrated a denial-of-service-style attack pattern. Wireshark captures showed a high volume of TCP SYN packets without completion of the three-way handshake, confirming the presence of half-open connections and automated flooding behaviour. This illustrates how resource exhaustion attacks can be detected through traffic volume, packet flags, and connection patterns.

Overall, this project reinforced the significance of continuous network monitoring and traffic analysis in cybersecurity. Understanding how normal traffic differs from reconnaissance and attack traffic enables security professionals to detect threats early, respond effectively, and strengthen defensive mechanisms such as intrusion detection and prevention systems.